



UNIVERSITI MALAYSIA PAHANG AL-SULTAN ABDULLAH

UNIVERSITI MALAYSIA PAHANG AL-SULTAN ABDULLAH

CENTRE OF MATHEMATICS SCIENCES

BSD2213 DATA SCIENCE PROGRAMMING 1

**GROUP PROJECT TITLE:
POMODORO TECHNIQUE TIMER**

LECTURER'S NAME: DR. NORAZIAH BINTI ADZHAR

SEMESTER 1 SESSION 2023/2024

GROUP MEMBERS

NO	NAME	MATRIC NO
1.	MUHAMMAD BIN ABDUL HADI	SD22050
2.	NURUL FAQIHAH BINTI MAZLI AMRAN	SD22047
3.	AQILAH MAISARAH BINTI AZIZI	SD22013
4.	NURUL HIDAYAH BINTI ROSLAN	SD22052

TABLE OF CONTENT

1.0 Introduction.	3
2.0 Why this project?	4
3.0 How can this project be extended?	5
4.0 Source Code	6
5.0 GUI Screenshot	46
6.0 References	57

1.0 Introduction.

The Pomodoro Timer is a revolutionary time management tool designed to enhance productivity and focus. Originating from the Pomodoro Technique, this timer encourages individuals to break their work into intervals, traditionally 25 minutes in length, separated by 5 minutes short breaks. While various Pomodoro timers exist, there is a distinct need for a specialized system tailored to the unique requirements of University of Malaysia Pahang Al-Sultan Abdullah (UMPSA) students. Recognizing the challenges faced by students in managing their study sessions efficiently, such as distractions and burnout, we have developed a Pomodoro Timer specifically catering to the academic needs of UMP students. This tool aims to provide an effective and user-friendly solution, empowering students to optimize their study sessions, improve concentration, and ultimately achieve academic success.

2.0 Why this project?

By developing a Pomodoro timer app, it is a project that engages us with the student directly relevant to their academic journey. The Pomodoro technique is a tried-and-true strategy for increasing productivity and focus. It especially connects with the difficulties students frequently encounter in efficiently organizing their study time. Students can customize their study sessions to suit their own tastes and patterns of focus by creating an app that is easy to use and offers customizable settings and customizable timing intervals.

The Pomodoro approach discourages multitasking by emphasizing the need to minimize distractions to stay focused on the work at hand. Even though a lot of us take pride in our multitasking skills, studies show that multitasking has negative impacts on the brain and reduces productivity overall. According to a University of California, Irvine research, it takes an average of 23 minutes and 15 seconds to refocus after changing tasks. This time nearly exactly corresponds to a single Pomodoro. Understanding how working intentionally during a Pomodoro reduces interruptions and multitasking gives another level of insight into why this productivity method works so well.

To sum up, students find the Pomodoro timer app very helpful as it helps them with their time management issues. Adaptable features allow for a customized study strategy, and complying to the principles of the Pomodoro technique reduces multitasking and distractions. This encourages better focus during study periods and complements leisure activities by suggesting quick games and methods of relaxation during breaks. These interesting breaks not only provide a welcome mental refreshment but also enhance the effectiveness and enjoyment of studying. This app turns into a useful tool that improves focus, optimizes study techniques, and helps students succeed academically overall.

3.0 How can this project be extended?

We've upgraded the Pomodoro Timer by including some useful features. To help you focus and unwind, we've included options for relaxing white noise in addition to the countdown. During your timed session, picture yourself listening to music such as Study Music by Greenred Productions, Background Music for Concentration, or Focus Music for Work and Studying on YouTube. We've put in small activities here and there to provide a little entertainment value and relaxation. After studying, taking a quick break to play a game is a great way to clear your head and feel more energized. It's crucial to set limits on game time and swiftly return to your studies to keep focused on your learning objectives and stay on track.

We included inspirational and uplifting videos to watch during the breaks. These movies highlight the successes, difficulties, or life experiences of people in your academic or professional sector. Watching them will give you a huge confidence and motivation boost, enabling you to go towards your goals with new energy. You can reset the timer and begin a fresh session after it has finished counting down. This function makes sure that our program is reusable, adjusting to your speed and letting you set aside more time if necessary to finish your task. Our aim is to provide a comprehensive tool that will increase your productivity while also improving the quality and enjoyment of your study sessions.

Apart from the mentioned, we've added a considerate element to your leisure time. We are aware that taking breaks is an opportunity for refreshment and relaxation. We've included suggested activities for people to think about during their breaks because of this. For example, going for a little walk is one of the recommended exercises. A little walk helps stimulate your senses and calm your head in addition to its physical benefits. This easy-to-do but powerful exercise will help you focus better when you go back to your studies. By making these recommendations, we hope to make sure that your break time turns into intentional time for recharging rather than merely a stop in your work, encouraging a more balanced and healthy approach to your studies session.

4.0 Source Code

```
import os

import tkinter as tk

from tkinter import ttk, messagebox

from tkinter import Tk, Toplevel, Label, Button

from tkinter.simpledialog import askstring

from PIL import Image, ImageTk

import pygame.mixer

import pygame

import sys

import random

import vlc

class VideoPlayerApp:

    def __init__(self, root):

        self.root = root

        self.root.title("Video Player")

        self.destroyed = False

        self.file_path = r"C:\Users\Qlhmysrh\Downloads\motivationalvid.mp4"

        self.video_duration = 10 # 3 minutes

        self.video_window = Toplevel(self.root)

        self.video_window.title("Video Player")
```

```

self.canvas = tk.Canvas(self.video_window, bg="black")

self.canvas.pack(expand=True, fill="both")

self.timer_label = ttk.Label(self.video_window, font=('Helvetica', 20), background="black",
foreground="white")

self.timer_label.place(relx=0.5, rely=0.05, anchor="center")

self.vlc_instance = vlc.Instance("--no-xlib")

self.media_player = self.vlc_instance.media_player_new()

self.root.after(0, self.play_video)

```

```

def play_video(self):

```

```

    media = self.vlc_instance.media_new(self.file_path)

    media.get_mrl()

    self.media_player.set_media(media)

    self.media_player.set_hwnd(self.canvas.winfo_id())

    self.media_player.play()

    self.video_window.geometry("800x600")

    self.video_window.after(int(self.video_duration * 1000), lambda: self.cleanup())

    self.update_timer_label()

```

```

def update_timer_label(self, remaining_time=None):

```

```

    if remaining_time is None:

        remaining_time = self.video_duration

```

```

if remaining_time >= 0:

    minutes, seconds = divmod(remaining_time, 60)

    timer_text = f"Time Remaining: {minutes:02d}:{seconds:02d}"

    self.timer_label.config(text=timer_text)

    self.root.after(1000, lambda: self.update_timer_label(remaining_time - 1))

    self.timer_label.place(relx=0.5, rely=0.95, anchor="center")


def cleanup(self):

    if not self.destroyed:

        self.media_player.stop()

        self.media_player.release()

        response = messagebox.askokcancel("Continue Studying", "It's time to continue
studying! Press 'OK' to acknowledge.")

        if response:

            self.root.deiconify()

            self.video_window.destroy()

            self.destroyed = True


def stop_video(self):

    if self.media_player is not None:

        self.media_player.stop()


class PictureViewerApp:

```

```
def __init__(self, master, image_paths):

    self.master = master

    self.image_paths = image_paths

    self.current_index = 0

    self.image = Image.open(self.image_paths[self.current_index])

    self.photo = ImageTk.PhotoImage(self.image)

    # Set background color for the main page

    self.master.configure(bg='lightblue')

    # Create and configure widgets

    self.button = tk.Button(master, text="Here is the suggestion", command=self.start_study,
                             font=('Arial', 16),
                             padx=20)
```

```
self.button.pack(pady=350)

self.label = tk.Label(master)

self.timer_label = tk.Label(master, text="")

self.timer_label.pack()

self.remaining_time = 2 # 5 minutes in seconds

def start_study(self):

    # Hide the button

    self.button.pack_forget()

    # Load initial image

    self.current_index = 0

    self.image = Image.open(self.image_paths[self.current_index])

    self.photo = ImageTk.PhotoImage(self.image)
```

```
# Set up the timer

self.update_timer()

# Display the image

self.label.configure(image=self.photo)

self.label.image = self.photo

self.label.pack()

def update_timer(self):

    minutes, seconds = divmod(self.remaining_time, 60)

    self.timer_label.config(text=f"Time Remaining: {minutes:02d}:{seconds:02d}")

    if self.remaining_time > 0:

        self.remaining_time -= 1
```

```

        self.master.after(1000, self.update_timer)

    else:

        self.show_notification()

def show_notification(self):

    # Display a notification after 5 minutes

    messagebox.showinfo("Study Reminder", "It's time to continue studying! Press 'OK' to
    acknowledge.")

    # Close the suggestion display

    self.master.destroy()

class CustomShapedButton(tk.Canvas):

    def __init__(self, master=None, image_path="", command=None, width=30, height=30,
    **kwargs):

        tk.Canvas.__init__(self, master, width=width, height=height, **kwargs)

        self.button = ttk.Button(self, text="", command=command)

```

```
self.create_window(width / 2, height / 2, window=self.button, anchor="center") # Adjusted  
placement
```

```
if image_path:
```

```
    self.button_image = Image.open(image_path).resize((width, height))
```

```
    self.button_photo = ImageTk.PhotoImage(self.button_image)
```

```
    self.config(width=self.button_image.width, height=self.button_image.height)
```

```
    self.button.config(image=self.button_photo, compound="center")
```

```
class StudyPomodoroApp:
```

```
    def __init__(self, root, username):
```

```
        self.root = root
```

```
        self.root.title("Study Pomodoro Timer")
```

```
        self.background_photo = None
```

```
        self.video_app = None
```

```
        pygame.mixer.init()
```

```
        self.mixer = pygame.mixer
```

```
        # Timer variables
```

```
        self.timer_running = False
```

```
        self.remaining_time = 5 # 25 minutes in seconds
```

```
        self.total_elapsed_time = 0 # Total elapsed time in seconds
```

```
        self.cycles_completed = 0
```

```

self.total_cycles = 4

self.cycle_duration = 5 # 25 minutes in seconds

self.break_duration = 3 # 5 minutes in seconds

# Additional variable to track the game state

self.game_over = False


# Study Timer Label with a creative font

self.timer_label = ttk.Label(root, text="25:00", font=("Comic Sans MS", 48, "italic",
"bold"))

self.timer_label.place(relx=0.5, rely=0.4, anchor="center")


# Buttons

ttk.Button(root, text="Start", command=self.start_timer).place(relx=0.3, rely=0.6,
anchor="center")

ttk.Button(root, text="Stop", command=self.stop_timer).place(relx=0.5, rely=0.6,
anchor="center")

ttk.Button(root, text="Reset", command=self.reset_timer).place(relx=0.7, rely=0.6,
anchor="center")


# Custom-shaped White Noise Playlist Button

white_noise_button_image_path = os.path.join(

    r"C:\Users\Qlhmysrh\OneDrive\Pictures\b41d094c54d3949b32a8592924933ff3.jpg")

white_noise_button_image = Image.open(white_noise_button_image_path).resize((30, 30))

white_noise_button_photo = ImageTk.PhotoImage(white_noise_button_image)

```

```

custom_button = CustomShapedButton(root, image_path=white_noise_button_image_path,
                                   command=self.show_white_noise_playlist, width=30, height=30,
bd=0,

                                   highlightthickness=0)

custom_button.create_image(0, 0, anchor="nw", image=white_noise_button_photo)

custom_button.image = white_noise_button_photo

custom_button.place(relx=0.5, rely=0.9, anchor="center")

# ttk.Button(root, text="Take a Break", command=self.take_break).place(relx=0.5,
rely=0.75, anchor="center")


# Username label

self.username_label = ttk.Label(root, text=f"Welcome, {username}!", font=("calibri", 16))


def start_timer(self):

    if not self.timer_running:

        self.timer_running = True

        self.update_timer()


def stop_timer(self):

    self.timer_running = False


def reset_timer(self):

```

```

self.timer_running = False

self.remaining_time = 1500 # Reset to 25 minutes

self.update_timer()

def resume_timer_after_video(self):

    # Implement the logic to resume the main timer after the video player is closed

    # For example, set the remaining time and call the update_timer method

    self.remaining_time = self.cycle_duration # Set the remaining time for the next cycle

    self.timer_running = True

    self.update_timer()

def play_next_audio(self, listbox, title_label, white_noise_options):

    # Get the indices of the currently selected items

    current_indices = listbox.curselection()

    # Check if any item is selected

    if current_indices:

        # Get the first index in the tuple

        current_index = current_indices[0]

        # Calculate the index of the next item, looping back to the beginning if necessary

        next_index = (current_index + 1) % listbox.size()

```

```

# Clear the current selection

listbox.selection_clear(0, tk.END)


# Set the selection to the next item

listbox.selection_set(next_index)

listbox.activate(next_index)


# Get the path of the selected audio file and play it

selected_audio_path = self.get_selected_audio_path(listbox)

if selected_audio_path:

    # Stop any currently playing sound

    self.mixer.stop()


    # Play the new audio

    self.play_audio(selected_audio_path, title_label)

else:

    # If no item is selected, you can decide what action to take (e.g., display a message)

    messagebox.showinfo("No Selection", "Please select an item before pressing Next.")


def get_selected_audio_path(self, listbox):

    selected_index = listbox.curselection()

    if selected_index:

        selected_item = listbox.get(selected_index)

```

```

print(f"Selected item: {selected_item}") # Add this line for debugging

white_noise_options = {

    "Rainforest": os.path.join(r"C:\Users\Qlhmysrh\Downloads\Rainforest music.mp3"),

    "Cafe Music": os.path.join(r"C:\Users\Qlhmysrh\Downloads\Cafe music.mp3"),

    "Ocean Music": os.path.join(r"C:\Users\Qlhmysrh\Downloads\Ocean music.mp3")

}

return white_noise_options.get(selected_item, "")

return ""

def play_audio(self, audio_path, title_label):

    # Stop any currently playing sound

    self.mixer.stop()

    # Load and play the new sound

    sound = self.mixer.Sound(audio_path)

    sound.play()

    # Update the title label with the name of the currently playing audio

    title_label.config(text=os.path.basename(audio_path))

def update_timer(self):

    if self.timer_running and self.remaining_time > 0:

        mins, secs = divmod(self.remaining_time, 60)

```

```

time_format = "{:02d}:{:02d}".format(mins, secs)

self.timer_label.configure(text=time_format)

self.remaining_time -= 1

self.total_elapsed_time += 1


# Check if the study session is completed

if self.remaining_time == 0 and self.total_elapsed_time % self.cycle_duration == 0:

    self.timer_running = False

    self.cycles_completed += 1


if self.cycles_completed < self.total_cycles:

    # Start the break after each study session

    self.show_break_notification()

else:

    # All cycles completed, reset the counters and show the last page

    self.cycles_completed = 0

    self.total_elapsed_time = 0

    self.show_last_page()


else:

    self.root.after(1000, self.update_timer)

else:

    # Timer is not running, handle other cases if needed

```

```

        self.root.deiconify()

# Add a new method to resume the timer after the suggestion tab is closed
def resume_timer_after_suggestion(self):

    self.remaining_time = self.cycle_duration

    self.timer_running = True

    self.update_timer()

def show_break_notification(self):

    # Create a new window for break activity selection

    break_activity_window = Toplevel(self.root)

    break_activity_window.title("Choose Break Activity")

    # Create buttons for break activities

    ttk.Button(break_activity_window, text="Activity Suggestion",

               command=lambda: self.set_break_activity("Activity Suggestion",
break_activity_window)).pack()

    ttk.Button(break_activity_window, text="Watch a Video",

               command=lambda: self.set_break_activity("Watch a Video",
break_activity_window)).pack()

    ttk.Button(break_activity_window, text="Play a Game",

               command=lambda: self.play_game(break_activity_window)).pack()

    ttk.Button(break_activity_window, text="Free Break",

```

```
        command=lambda: self.set_break_activity("Free Break",
break_activity_window)).pack()
```

```
def set_break_activity(self, break_activity, break_activity_window):
```

```
    # Close the break activity window
```

```
    break_activity_window.destroy()
```

```
    # Handle the selected break activity
```

```
    if break_activity == "Play a Game":
```

```
        self.play_game()
```

```
    elif break_activity == "Watch a Video":
```

```
        self.watch_video()
```

```
    elif break_activity == "Activity Suggestion":
```

```
        self.suggestion()
```

```
    else:
```

```
        # Handle other break activities
```

```
        messagebox.showinfo("Take a Break", f"It's time for a 5-minute break!\nBreak Activity:
{break_activity}")
```

```
        self.remaining_time = self.break_duration
```

```
        self.timer_running = True
```

```
        self.update_timer_after_break()
```

```

# Check if all break activities are completed

if self.cycles_completed >= self.total_cycles:

    # All cycles completed, reset the counters

    self.cycles_completed = 0

    self.total_elapsed_time = 0

    self.show_last_page()

    self.remaining_time = self.cycle_duration

    self.timer_running = True

    self.update_timer()


# Restore the main StudyPomodoroApp window

self.root.deiconify()


def watch_video(self):

    # Create and save the VideoPlayerApp instance

    self.video_app = VideoPlayerApp(self.root)


# Hide the main StudyPomodoroApp window during the break activity

self.root.withdraw()


# Check if the video window is closed and resume the timer afterward

def video_player_loop():

    # Check if the main window is destroyed (application closed)

```

```

if not self.root.winfo_exists():

    return

# Check if the video window is closed

if not self.video_app or self.video_app.destroyed:

    # Resume the timer after the video window is closed

    self.resume_timer_after_video()

    self.root.deiconify()

else:

    # Continue the loop

    self.root.after(100, video_player_loop)

# Start the video player loop

video_player_loop()

def suggestion(self):

    # Stop the timer before displaying the suggestion window

    self.timer_running = False

# Provided image paths

picture_paths = [r"C:\Users\Qlhmysrh\Downloads\Pomodoro Study System (1).png"]

# Create a new Tkinter window for the PictureViewerApp

```

```

break_activity_window = tk.Toplevel(self.root)

break_activity_window.title("Activity Suggestion")


# Initialize PictureViewerApp with the provided image paths
picture_viewer_app = PictureViewerApp(break_activity_window, picture_paths)


# Hide the main StudyPomodoroApp window during the break activity
self.root.withdraw()


# Define a function to restore the main window when the suggestion window is closed
def on_break_activity_window_close():

    break_activity_window.destroy()

    self.root.deiconify()

    # Resume the timer countdown after the 'suggestion' display ends
    self.resume_timer_after_suggestion()


# Set the 'on_break_activity_window_close' function to be called when the suggestion
window is closed

break_activity_window.protocol("WM_DELETE_WINDOW",
on_break_activity_window_close)


def suggestion_loop():

    # Check if the main window is destroyed (application closed)

```

```

    if not self.root.winfo_exists():

        return

    # Check if the suggestion window is closed

    if not break_activity_window.winfo_exists():

        # Resume the timer after the suggestion window is closed

        self.resume_timer_after_suggestion()

        return

    # Continue the loop

    self.root.after(100, suggestion_loop)


# Start the suggestion loop

suggestion_loop()


def play_game(self, break_activity_window):

    # Execute the Snake game code here

    pygame.init()


    sw, sh = 800, 800


    # Get the width and height of the user's display

    user_screen_info = pygame.display.Info()

    sw, sh = user_screen_info.current_w, user_screen_info.current_h

```

```
block_size = 50
```

```
font_style = pygame.font.SysFont(None, 30)
```

```
screen = pygame.display.set_mode((sw, sh), pygame.FULLSCREEN)
```

```
pygame.display.set_caption("Snake!")
```

```
clock = pygame.time.Clock()
```

```
class Snake:
```

```
    def __init__(self):
```

```
        self.x, self.y = block_size, block_size
```

```
        self.xdir = 1
```

```
        self.ydir = 0
```

```
        self.head = pygame.Rect(self.x, self.y, block_size, block_size)
```

```
        self.body = [pygame.Rect(self.x - block_size, self.y, block_size, block_size)]
```

```
        self.dead = False
```

```
    def update(self):
```

```
        global apple
```

```
        for square in self.body:
```

```
            if self.head.colliderect(square):
```

```
                self.dead = True
```

```
            if not (0 <= self.head.x < sw and 0 <= self.head.y < sh):
```

```

        self.dead = True

    if self.dead:

        self.__init__() # Reset snake when dead

        apple = Apple()

    self.body.append(self.head.copy()) # Use a copy to avoid reference issues

    if len(self.body) > 1:

        self.body.pop(0) # Remove the tail

    self.head.x += self.xdir * block_size

    self.head.y += self.ydir * block_size


class Apple:

    def __init__(self):

        self.x = int(random.randint(0, sw) / block_size) * block_size

        self.y = int(random.randint(0, sh) / block_size) * block_size

        self.rect = pygame.Rect(self.x, self.y, block_size, block_size)

    def update(self):

        pygame.draw.rect(screen, "orange", self.rect)

    def drawGrid():

        for x in range(0, sw, block_size):

```

```

        for y in range(0, sh, block_size):

            rect = pygame.Rect(x, y, block_size, block_size)

            pygame.draw.rect(screen, "#3c3c3b", rect, 1)


def quit_game():

    pygame.quit()

    sys.exit()


score = font_style.render("1", True, "white")

score_rect = score.get_rect(center=(sw / 2, sh / 20))


# Display instructions before starting the game

instruction_text = [

    " Welcome to Snake!",

    " Use arrow keys to control the snake.",

    " Eat the orange apple to grow.",

    " Avoid colliding with the snake's body or the screen edges.",

    " You can press 'P' to pause the game",

    " You have 5 minutes to play.",

    " You can press 'Q' to Quit the game",

    " Press any key to start!"

]

```

```

instruction_rects = []

for i, line in enumerate(instruction_text):

    line_render = font_style.render(line, True, "white")

    line_rect = line_render.get_rect(center=(sw / 2, sh / 2 + i * 30))

    instruction_rects.append((line_render, line_rect))

for instruction_render, instruction_rect in instruction_rects:

    screen.blit(instruction_render, instruction_rect.topleft)

pygame.display.update()

# Wait for any key press to start the game

wait_for_key = True

while wait_for_key:

    for event in pygame.event.get():

        if event.type == pygame.QUIT:

            pygame.quit()

            sys.exit()

        if event.type == pygame.KEYDOWN:

            wait_for_key = False

# Reset variables for the actual game

start_time = pygame.time.get_ticks()

```

```

game_over = False

# Set up timer event

pygame.time.set_timer(pygame.USEREVENT, 1000) # Timer event every 1000
milliseconds (1 second)

snake = Snake()

apple = Apple()

# Initialize remaining_time outside the event loop

remaining_time = 300

pause = False # New variable to track the pause state

pause_start_time = 0 # New variable to track the time when the game was paused

quit_button = pygame.Rect(sw / 2 - 50, sh - 50, 100, 40)

while not game_over:

    for event in pygame.event.get():

        if event.type == pygame.QUIT:

            pygame.quit()

            quit_game()

        if event.type == pygame.KEYDOWN:

            if event.key == pygame.K_DOWN:

                snake.ydir = 1

                snake.xdir = 0

```

```

elif event.key == pygame.K_UP:

    snake.ydir = -1

    snake.xdir = 0

elif event.key == pygame.K_RIGHT:

    snake.ydir = 0

    snake.xdir = 1

elif event.key == pygame.K_LEFT:

    snake.ydir = 0

    snake.xdir = -1

elif event.key == pygame.K_p:

    if not pause:

        pause_start_time = pygame.time.get_ticks()

        pause = not pause

elif event.key == pygame.K_q: # Quit the game with the "Q" key

    pygame.quit()

    sys.exit()

elif event.type == pygame.MOUSEBUTTONDOWN:

    if quit_button.collidepoint(event.pos):

        quit_game()

if not pause: # Only update the game when not paused

    snake.update()

```

```

screen.fill('black')

drawGrid()

apple.update()

score = font_style.render(f"{len(snake.body) + 1}", True, "white")

pygame.draw.rect(screen, "green", snake.head)

for square in snake.body:
    pygame.draw.rect(screen, "green", square)

screen.blit(score, score_rect)

# Display remaining time
elapsed_time = (pygame.time.get_ticks() - start_time) / 1000
if elapsed_time > 3: # 300 seconds = 5 minutes
    game_over = True
else:
    remaining_time = int(300 - elapsed_time)

if snake.head.x == apple.x and snake.head.y == apple.y:
    snake.body.append(pygame.Rect(square.x, square.y, block_size, block_size))

```

```

apple = Apple()

# Display pause message
if pause:

    pause_text = font_style.render("Game Paused. Press 'P' to resume.", True, "white")

    pause_rect = pause_text.get_rect(center=(sw / 2, sh / 2))

    screen.blit(pause_text, pause_rect.topleft)

# Display remaining time only when not paused
if not pause:

    timer_text = font_style.render(f"Time Left: {remaining_time} seconds", True, "white")

    timer_rect = timer_text.get_rect(center=(sw / 2, sh - 20))

    screen.blit(timer_text, timer_rect.topleft)

    # Draw the Quit button

    pygame.draw.rect(screen, "red", quit_button)

    quit_text = font_style.render("Quit", True, "white")

    quit_rect = quit_text.get_rect(center=quit_button.center)

    screen.blit(quit_text, quit_rect.topleft)

pygame.display.update()

clock.tick(5)

# Update remaining time when the game is paused

```

```

if pause:

    pause_end_time = pygame.time.get_ticks()

    pause_duration = (pause_end_time - pause_start_time) / 1000

    remaining_time = max(0, remaining_time - pause_duration)

if remaining_time == 0:

    game_over = True

# Signal the end of the game to resume the study timer

self.resume_study_timer_after_game()

if self.game_over:

    print("Game Over! You exceeded the time limit of 5 minutes. Get back to your study.")


# Add a notification with a button

notification_font = pygame.font.SysFont(None, 40)

notification_text = notification_font.render("You need to get back to your study", True,
"white")

notification_rect = notification_text.get_rect(center=(sw / 2, sh / 2 - 50))


okay_button = pygame.Rect(sw / 2 - 50, sh / 2 + 50, 100, 50)


while True:

    for event in pygame.event.get():

        if event.type == pygame.QUIT:

            pygame.quit()

```

```

        sys.exit()

    if event.type == pygame.MOUSEBUTTONDOWN:

        if okay_button.collidepoint(event.pos):

            pygame.quit()

            sys.exit()

    screen.fill('black')

    screen.blit(notification_text, notification_rect.topleft)

    pygame.draw.rect(screen, "blue", okay_button)

    okay_text = font_style.render("Okay", True, "white")

    okay_rect = okay_text.get_rect(center=okay_button.center)

    screen.blit(okay_text, okay_rect.topleft)

    pygame.display.update()

    clock.tick(30) # Adjust the frame rate

    pygame.quit()

    sys.exit()

pygame.quit()

# Instead of calling sys.exit(), you can destroy the break_activity_window

break_activity_window.destroy()

def resume_study_timer_after_game(self):

```

```

# Additional logic to resume the study timer

# For example, you can update the study timer variables and call the update_timer method.

self.remaining_time = self.cycle_duration

self.timer_running = True

self.update_timer()

# Display a message when the game is over


def ask_for_break_activity(self):

    options = ["Free Break", "Watch a Video", "Play a Game"]

    break_activity = askstring("Break Activity",

                               "Choose a break activity:\n1. Free Break\n2. Watch a Video\n3. Play a
Game",

                               initialvalue=options[0])

    if break_activity in options:

        return break_activity

    else:

        return options[0]


def stop_activities(self):

    # Add logic to stop all ongoing activities

    pygame.mixer.stop() # Stop playing audio

    pygame.quit() # Quit Pygame to stop the game

```

```

def stop_game(self):

    # Add logic to stop the game

    pygame.mixer.stop() # Stop playing audio

    pygame.quit() # Quit Pygame to stop the game


def update_timer_after_break(self):

    if self.timer_running and self.remaining_time > 0:

        mins, secs = divmod(self.remaining_time, 60)

        time_format = "{:02d}:{:02d}".format(mins, secs)

        self.timer_label.configure(text=time_format)

        self.remaining_time -= 1

        self.root.after(1000, self.update_timer_after_break)

    elif self.remaining_time == 0:

        if self.cycles_completed < self.total_cycles:

            # Continue with the next study session (including break)

            self.total_elapsed_time = 0

            self.remaining_time = self.cycle_duration

            self.timer_running = True

            self.update_timer()

        # Increment cycles_completed only after both study and break

```

```

        self.cycles_completed += 0

        # Display cycle notification

        cycle_number = self.cycles_completed

        messagebox.showinfo("Cycle Notification", f"You completed Cycle {cycle_number}")

    else:

        # All cycles completed, reset the counters

        self.cycles_completed = 0

        self.total_elapsed_time = 0

        self.show_last_page()

    elif self.timer_running and self.remaining_time <= 0:

        # Timer is still running, but the remaining time is zero, show last page

        self.show_last_page()

    else:

        # Timer is not running, do something else if needed

        pass

def show_last_page(self):

    print("Entering show_last_page function")

    global background_photo # Declare as global

    last_page = Toplevel(self.root) # Use Toplevel instead of Tk

    last_page.title("Session Ended")

```

```

# Load the background image without resizing (Replace the path with your actual image
path)

background_image = Image.open(r"C:\Users\Qlhmysrh\Downloads\background timer.jpg")

background_photo = ImageTk.PhotoImage(background_image)

# Create a Label to display the background image

background_label = Label(last_page, image=background_photo)

background_label.place(x=0, y=0, relwidth=1, relheight=1) # Set the position and size


ttk.Label(last_page, text="YOUR SESSION HAS ENDED", font=("Comic Sans MS", 32,
"bold")).pack(pady=30)

ttk.Label(last_page, text="Congratulations, you did well! 🎉", font=("Comic Sans MS",
24)).pack(pady=20)

ttk.Label(last_page, text="Thank you for your service! 🙏", font=("Show Gothic",
16)).pack(pady=10)

restart_button = ttk.Button(last_page, text="Restart", command=self.confirm_restart)

restart_button.pack(pady=20)

end_button = ttk.Button(last_page, text="End", command=lambda:
self.end_program(last_page))

end_button.pack(pady=20)

```

```

print("Exiting show_last_page function")

# After the last page is shown, reset the video_app attribute
self.video_app = None

def end_program(self, last_page):
    last_page.destroy()

    self.root.destroy() # Destroy the main Tk() instance to end the program

def confirm_restart(self):
    result = messagebox.askokcancel("Confirmation", "Do you wish to continue?")

    if result:
        # Reset variables and restart the timer
        self.timer_running = False
        self.cycles_completed = 0
        self.total_elapsed_time = 0
        self.remaining_time = self.cycle_duration
        self.update_timer()
        print("Restarting...")

def show_white_noise_playlist(self):
    # Create a new window for the white noise playlist
    playlist_window = tk.Toplevel(self.root)

```

```

playlist_window.title("White Noise Playlist")

# Create a listbox to display the white noise options

listbox = tk.Listbox(playlist_window)

listbox.pack(padx=20, pady=20)

# Add white noise options to the listbox

white_noise_options = {
    "Rainforest": os.path.join(r"C:\Users\Qlhmysrh\Downloads\Rainforest music.mp3"),
    "Cafe Music": os.path.join(r"C:\Users\Qlhmysrh\Downloads\Ocean music.mp3"),
    "Ocean Music": os.path.join(r"C:\Users\Qlhmysrh\Downloads\Cafe music.mp3")
}

for option in white_noise_options:
    listbox.insert(tk.END, option)

# Add play, next, and stop buttons

title_label = ttk.Label(playlist_window, text="Now Playing: ", font=("calibri", 12))
title_label.pack()

play_button = CustomShapedButton(playlist_window,
                                  image_path=os.path.join(

```

```

r"C:\Users\Qlhmysrh\OneDrive\Pictures\b41d094c54d3949b32a8592924933ff3.jpg"),

        command=lambda:
self.play_audio(white_noise_options[listbox.get(tk.ACTIVE)],

                    title_label),

                    width=30, height=30)

play_button.place(relx=0.2, rely=0.85, anchor="center") # Adjusted placement


next_button = CustomShapedButton(

    playlist_window,

    image_path=os.path.join(r"C:\Users\Qlhmysrh\OneDrive\Pictures\next button.jpg"),

    command=lambda: self.play_next_audio(listbox, title_label, white_noise_options),

    width=30, height=30

)

next_button.place(relx=0.5, rely=0.85, anchor="center") # Adjusted placement


stop_button = CustomShapedButton(playlist_window,

    image_path=os.path.join(

        r"C:\Users\Qlhmysrh\OneDrive\Pictures\stop button.jpg"),

        command=self.stop_audio, width=30, height=30)

stop_button.place(relx=0.8, rely=0.85, anchor="center") # Adjusted placement


def stop_audio(self):

```

```
pygame.mixer.stop()
```

```
def get_user_name():
```

```
    username = askstring("User Name", "Enter your name:")
```

```
    if username:
```

```
        welcome_user(username)
```

```
def welcome_user(username):
```

```
    welcome_screen = Toplevel(main_screen)
```

```
    welcome_screen.title("Welcome")
```

```
    welcome_screen.geometry("300x250")
```

```
    Label(welcome_screen, text=f"Welcome, {username}!", font=("calibri", 25)).pack(side="top",  
pady=70)
```

```
    Button(welcome_screen, text="Continue", command=lambda: on_continue(username,  
welcome_screen)).pack(side="top",
```

```
pady=10)
```

```
def on_continue(username, welcome_screen):
```

```
    welcome_screen.destroy()
```

```

get_user_button.pack_forget() # Hide the "Get UserName" button

app = StudyPomodoroApp(main_screen, username)


def main_account_screen():

    global main_screen

    global get_user_button


    main_screen = Tk()

    main_screen.geometry("300x250")

    main_screen.title("Pomodoro Technique Timer")


    # background image

    image_path = os.path.abspath(r"C:\Users\Qlhmysrh\OneDrive\Pictures\stop button.jpg")


    try:

        background_image = Image.open(image_path)

        background_photo = ImageTk.PhotoImage(background_image)


        background_label = Label(main_screen, image=background_photo)

        background_label.place(x=0, y=0, relwidth=1, relheight=1)

```

```
Label(main_screen, text="Pomodoro Technique Timer", font=("Showcard Gothic",
30)).pack(side="top", pady=40)
```

```
get_user_button = Button(main_screen, text="Get User Name", height="3", width="30",
command=get_user_name)
```

```
get_user_button.pack(side="top", pady=50)
```

```
Label(main_screen, text="").pack()
```

```
main_screen.mainloop()
```

```
except FileNotFoundError:
```

```
    messagebox.showerror("File Not Found", f"The background image file was not found
at:\n{image_path}")
```

```
if __name__ == "__main__":
```

```
    # Start the application
```

```
    main_account_screen()
```

5.0 GUI Screenshot

Figure 1

Click to get username.



Figure 2

Users need to enter any username to start.



Figure 3
Welcome the user



Figure 4
Main timer page with button start, stop, reset, sound.



Figure 5

User can listen to any white noise playlist with play, next, pause button.

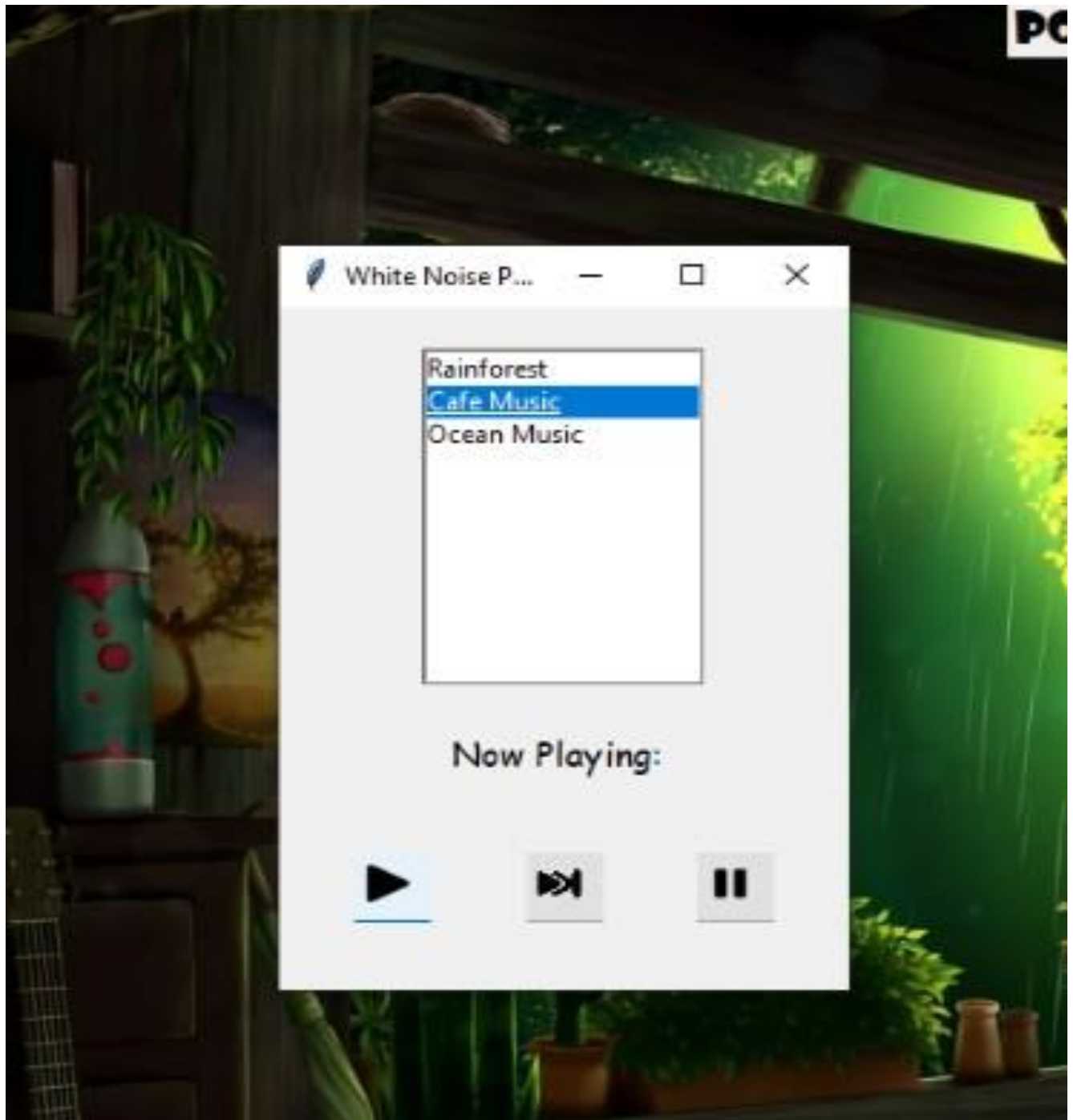


Figure 6

User can choose any option from the break time activity list.

Break Activity option: Watch a video

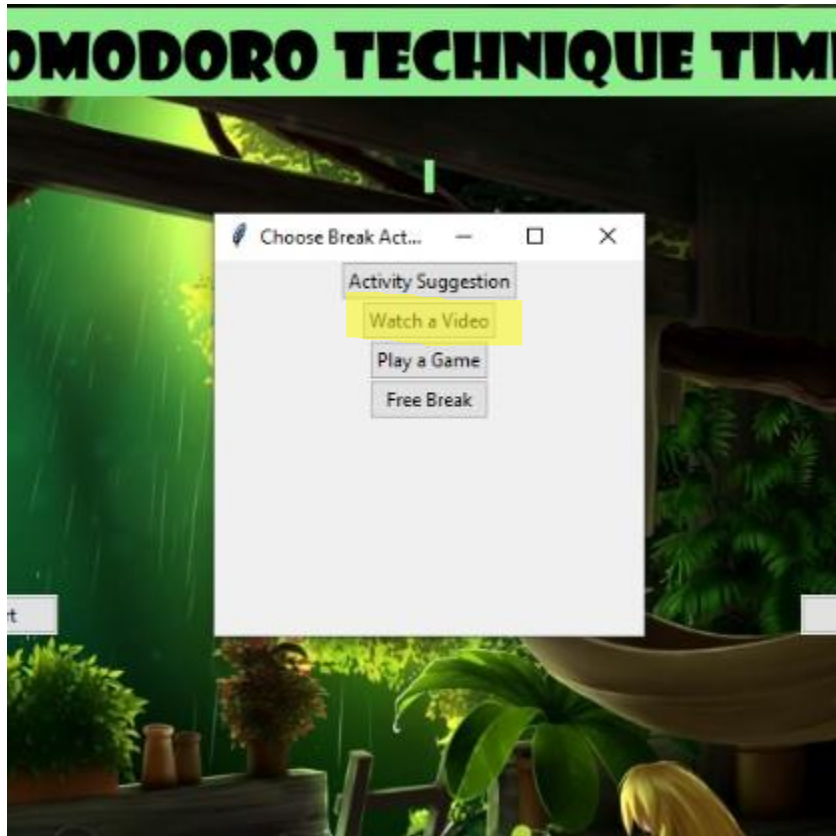


Figure 7

If user chooses to rest while watching video,user will be watching motivational video for 5 minutes.

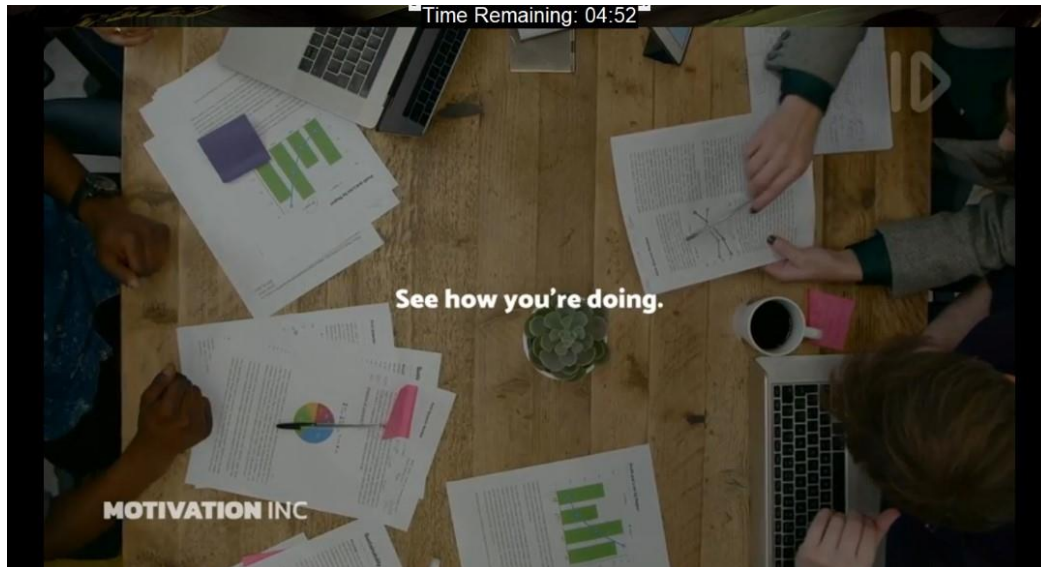


Figure 8

Break activity option: Activity Suggestion



Figure 9

If user wants a suggestion activity during their break time, it will appear creator suggestion and the benefit of it.

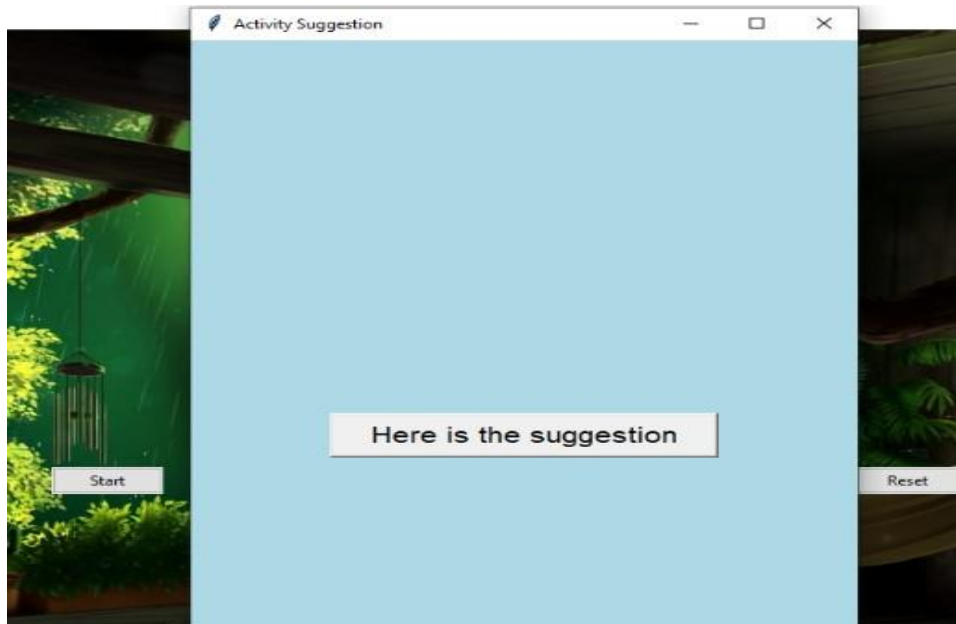


Figure 10

This is the suggested activity that you must do for 5 minutes break.

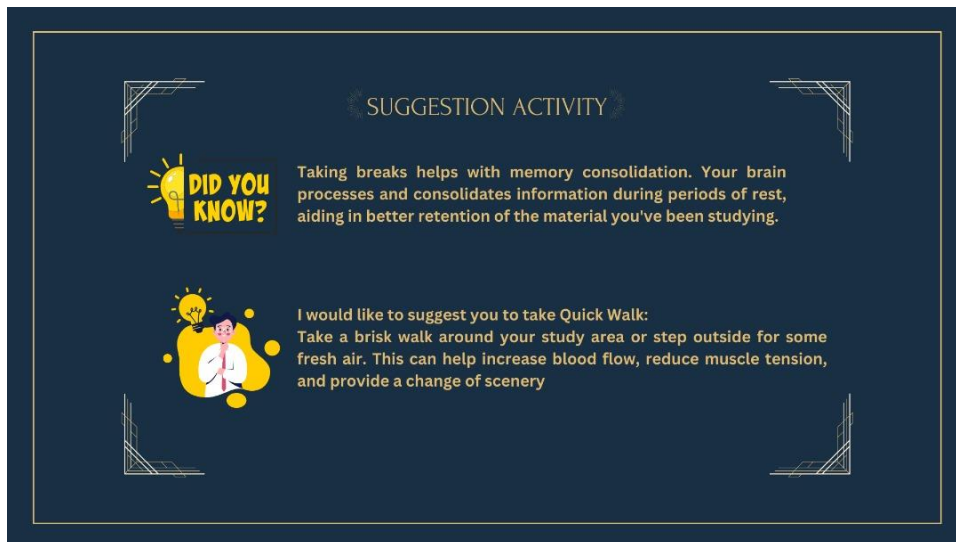


Figure 11

After 5 minutes, the reminder appeared to remind you to continue back their study.

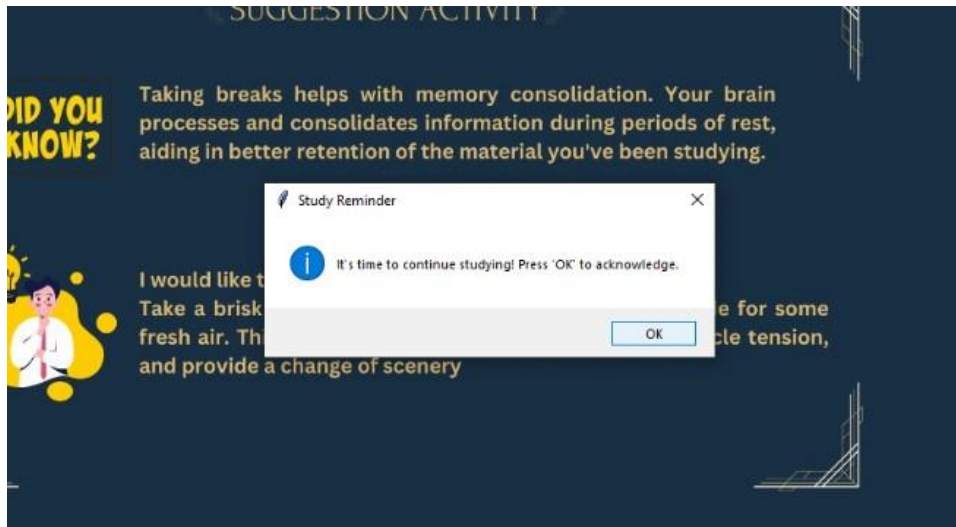


Figure 12

Break activity option : Free Break



Figure 13

If the user chooses to just take a break for 5 minutes, it will appear timer for 5 minutes of free break.

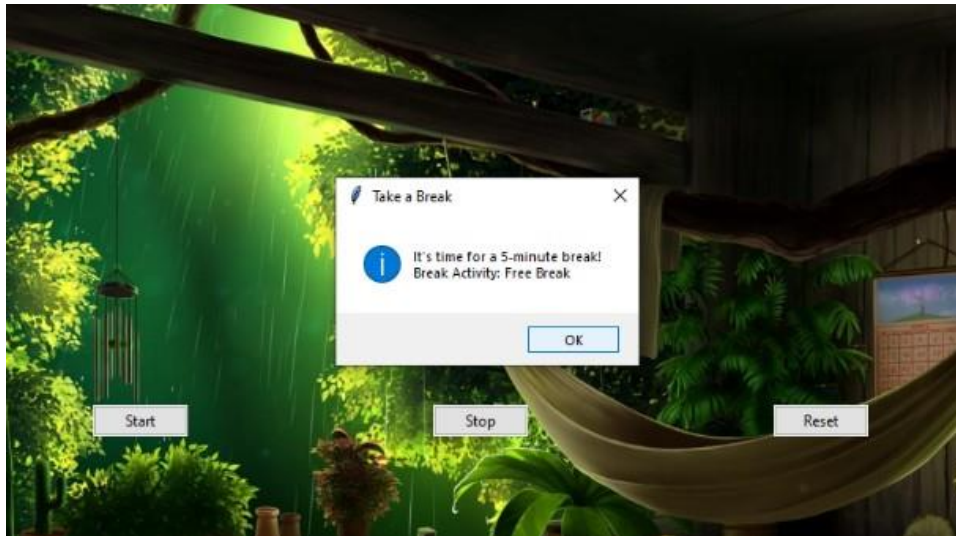


Figure 14

After 5 minutes of free break has ended, this notification is going to display to remind the user to go back studying.

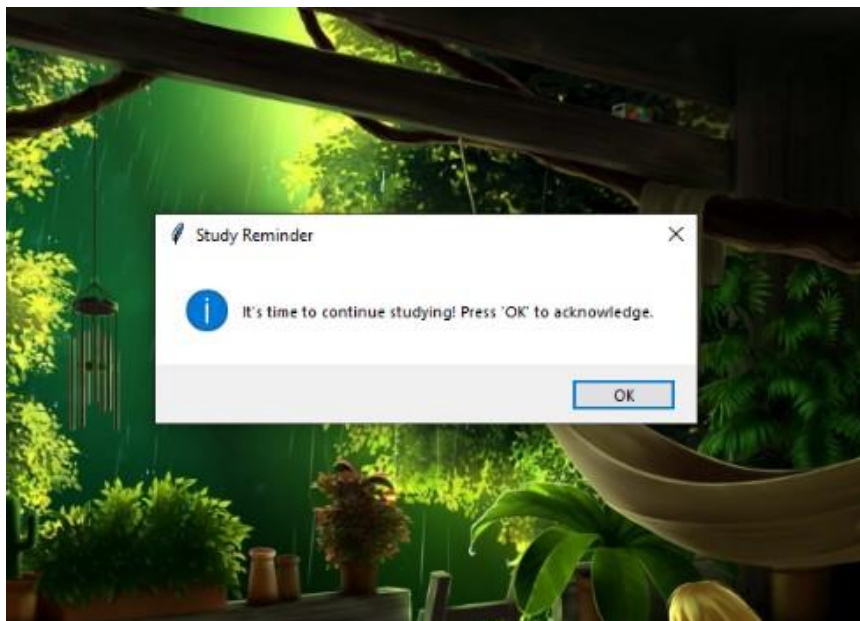


Figure 15

It will loop again at the main page timer.



Figure 16

Break activity option: play a game



Figure 17

If user chooses to play a game, it will appear timer for 5 minutes.

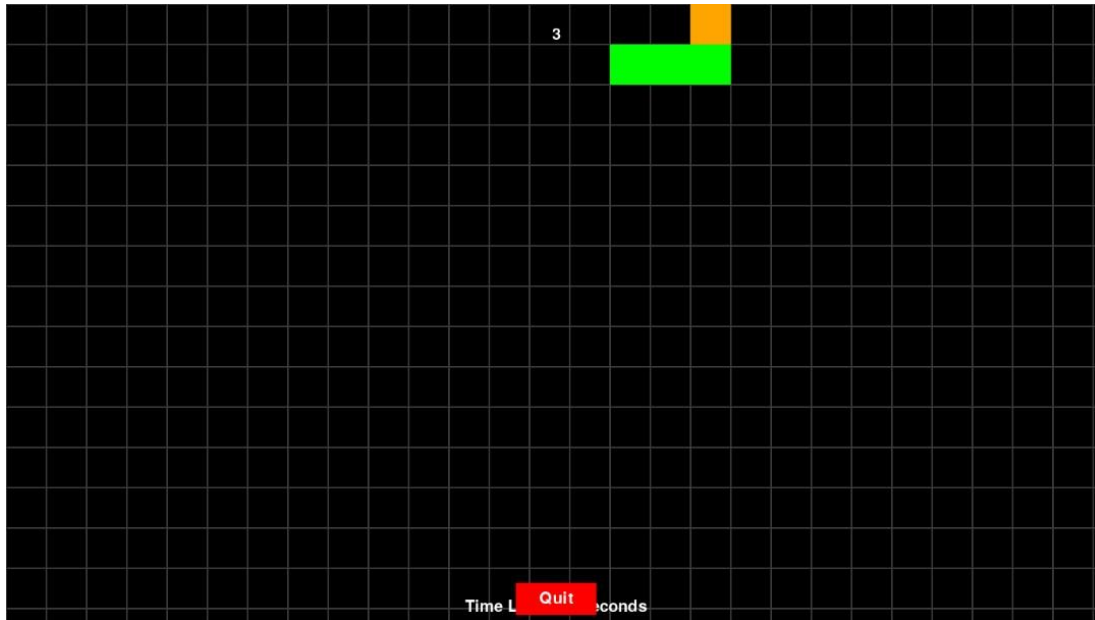


Figure 18

Finally, you study session has ended.

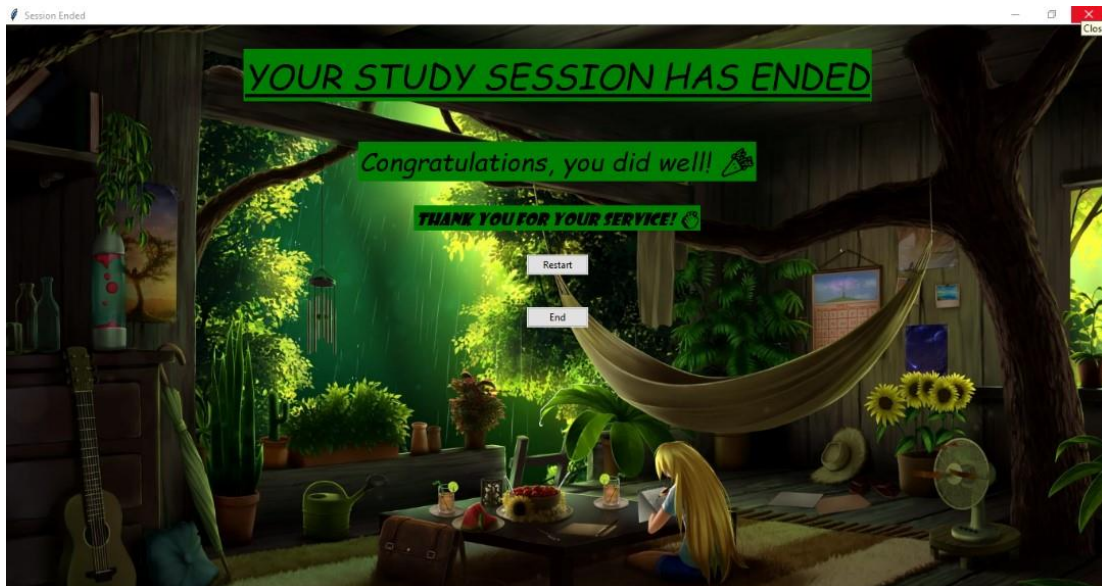
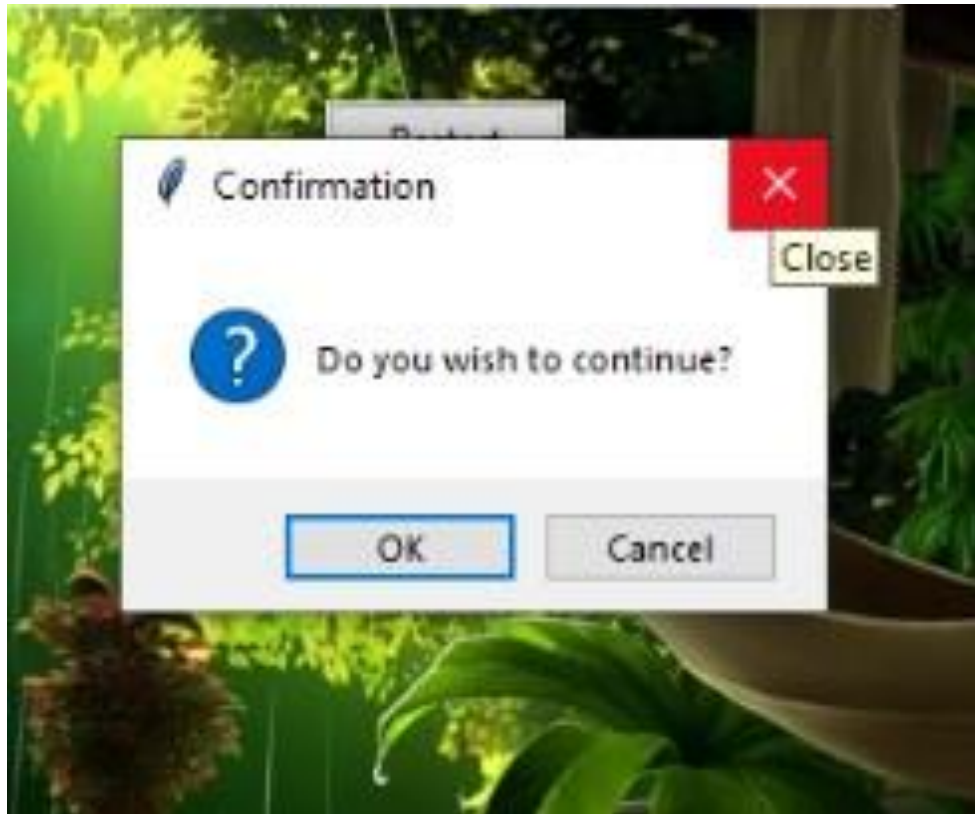


Figure 19

If user wants to continue use this application, click ok button.



6.0 References

<https://e-student.org/best-study-methods/>

<https://www.focusboosterapp.com/blog/the-science-behind-the-pomodoro-technique/>